DreamOS: Trustworthy Semantic File Management with Local Models and a Checkable Privacy Audit

Zaid Ali Syed Roll No. 2305139
Om Vyas Roll No. 2305170
Krushna Kadam Roll No. 2305165

Under the guidance of **Dr. Minakshi R. Rajput**

Department of Information Technology, Institute of Information and Communication Technology (IICT)
MGM University, Chhatrapati Sambhajinagar, India | sidzaid72@gmail.com

Abstract: Personal file collections are organised by names chosen at save time, and those names rarely describe the content later. Language-model assistants can search by meaning, but they can also launch the wrong file, follow a mistaken guess, or send data off the machine without the user knowing. We present DreamOS, a locally run desktop file manager that finds, opens, relates and organises files from natural-language requests using a local embedding model and a local 3B language model, with four explicit trust mechanisms: a confidence guard on the open action, evidence-before-model precedence for follow-ups, reversible organisation and a privacy audit that can be falsified. On 36 labelled queries over a 37-file vault, DreamOS ranks the intended file first for 94% of queries (MRR 0.97) against 56% for a BM25 content index and 33% for filename matching; on the 10 queries sharing no word with their target, both lexical baselines score 0.00 and DreamOS 0.80. Opening the top result unconditionally would launch 2 wrong files; the guarded action launches 29 files, all correct, at the cost of 5 extra confirmations. On synthetic collections of 500 to 5,000 files, replacing three all-pairs computations cut the graph rebuild from 143.3 s to 3.2 s at 5,000 files. Limitations, including the small hand-labelled query set, are stated in full.

Index Terms: trustworthy AI, semantic search, sentence embeddings, knowledge graph, local language models, privacy by design, responsible computing.

I. INTRODUCTION

A personal computer accumulates thousands of files whose names were chosen once, in a hurry, and rarely describe what the file is about. A user who saved final_final_v3.txt must later remember that exact name, or the folder it went into, to find it again. Mainstream desktop search is lexical: a query must share vocabulary with the file name, its metadata or its text [5]-[7]. Semantic file systems [1], [8] and dense retrieval [3], [9] show that matching by meaning is feasible, but the closest recent system, LSFS [1], is designed for software agents calling an API, and neither it nor the consumer tools report in a checkable way what could leave the user's machine.

Generative AI makes such an assistant easy to build and easy to distrust: a language model can route a request to the wrong action, an open request can launch the wrong file, and a claim that data stays local is usually only asserted. Trustworthy and responsible AI frameworks ask for systems whose behaviour is measured, bounded and explainable [10]. DreamOS is a desktop file manager built around that requirement. A person describes a file in plain language and has it found, opened, related to other files, or organised, with every inference step running on the user's own computer, and each place where the model could cause harm is guarded by a mechanism that is tested and measured. The contributions are:

1) An architecture that exposes semantic retrieval and action through one conversational interface for a human user, with six cooperating modules and four explicit trust mechanisms (Table III): a confidence guard on the open action, evidence-before-model precedence, reversible moves and a checkable privacy audit.
2) A controlled retrieval comparison against two lexical baselines on a labelled query set, partitioned by whether a query shares any word with its target, with confidence intervals and the failures listed.
3) A measurement of the guard: on the labelled queries, opening the top result unconditionally would launch two wrong files, while the guarded action launches 29 files, all correct, and blocks both, at the price of five unnecessary confirmations; and a deterministic-precedence rule for follow-ups, introduced after a live failure in which the model's own "refers to previous" flag overrode a confident search result.
4) A vectorised, equivalence-tested knowledge-graph construction with mean-centred similarity and average-linkage clustering, evaluated at up to 5,000 files with the real local models, in which profiling-driven fixes removed all-pairs computations and brought the 5,000-file rebuild to seconds.
5) A privacy audit that reports endpoints from live configuration, with an independent network-level check that can falsify it.

II. RELATED WORK

A. Semantic file systems and personal retrieval

Gifford et al. introduced semantic file systems, in which attributes extracted from content are exposed as virtual directories [8]. Stuff I've Seen made previously encountered personal information retrievable by content rather than location [11]. LSFS, presented at ICLR 2025, indexes files as embedding vectors and exposes create, retrieve, update and roll-up operations to an LLM agent, showing that embedding indexes can replace command-based file operations at the operating-system-agent level [1]. DreamOS follows the same architectural direction but targets a person rather than an agent, adds a relationship graph and conversational memory, and reports an auditable privacy statement.

B. Dense retrieval, anisotropy and ranking

Word and sentence embeddings place related text near each other [2], [3]; dense passage retrieval showed that learned embeddings can rival lexical ranking such as BM25 [9], [12]. Sentence-embedding spaces are anisotropic, so unrelated texts have high cosine similarity [13], [14]; DreamOS observes this directly (median document-pair cosine 0.52 on its vault) and counters it by mean-centring before graph construction, in the spirit of [14]. Approximate nearest-neighbour indexes such as HNSW [15] are what the vector store uses internally.

C. Consumer desktop search

Windows Search indexes names, metadata and text but matches lexically [5]; Spotlight layers ranking heuristics on a keyword index [6]; Everything indexes NTFS file names without reading content [7]. Table I summarises the gap: none retrieves by meaning in the documented configuration, and none reports a checkable statement of what leaves the machine.

TABLE I. Capability Comparison with Existing File-Search Systems

Capability	Windows Search [5]	Spotlight [6]	Everything [7]	LSFS [1]	DreamOS
Retrieval by embedding similarity (meaning)	No	No	No	Yes	Yes
Indexes file content	Yes	Yes	No (names)	Yes	Yes
Conversational requests with follow-ups	No	No	No	No (agent API)	Yes
Opens a file only above a confidence bar	No	No	No	Not assessed	Yes
Reversible AI-assisted organisation	No	No	No	Not assessed	Yes
Typed file-relationship graph with reasons	No	No	No	Not assessed	Yes
Needs no cloud model or service	Yes	Yes	Yes	Not shown	Yes
Live, checkable privacy report	No	No	No	No	Yes

"No" means the capability is not provided in the standard configuration documented in [5]-[7]; vendor features added after those documents (for example AI-assisted search on newer hardware) were not evaluated. LSFS was assessed from its published paper only; "Not assessed" marks properties that paper does not state.

III. SYSTEM DESIGN

A. Architecture

DreamOS is a client-server application confined to one machine (Fig. 1). A Tauri 2 shell hosts a React interface with four views. It calls a FastAPI service on 127.0.0.1:8420, which calls a local Ollama runtime for two models, nomic-embed-text [16] for embeddings and llama3.2 (3B) for intent, summaries and tags, and uses three local stores: SQLite for relational data, ChromaDB [4] for chunk vectors and a sandboxed vault of files.

B. Pipelines

Two pipelines share these components (Fig. 2). Indexing extracts text, chunks and embeds it, stores the vectors, asks the model for a summary, tags and category, and rebuilds the graph; unchanged files are skipped by content hash. The query pipeline classifies each message into one of five intents (search, open, related, workspace, organise), resolves references to earlier results, and routes the request.

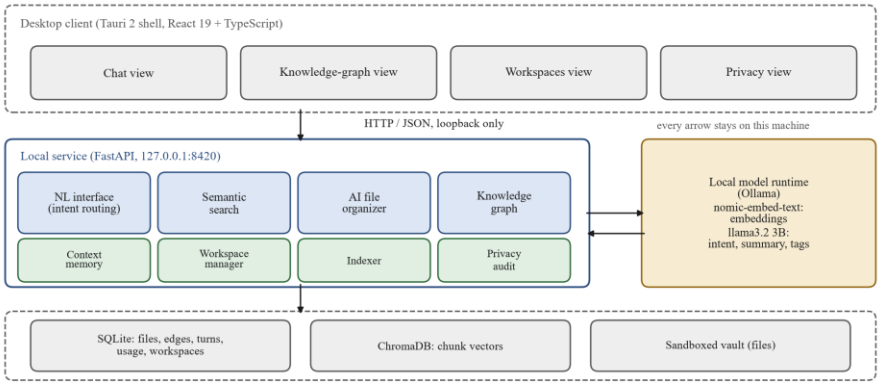


Fig. 1. DreamOS architecture. A desktop client talks over loopback HTTP to a local service with eight components (blue: request modules; green: supporting modules). The service calls a local model runtime and three local stores. Nothing in the path leaves the machine.

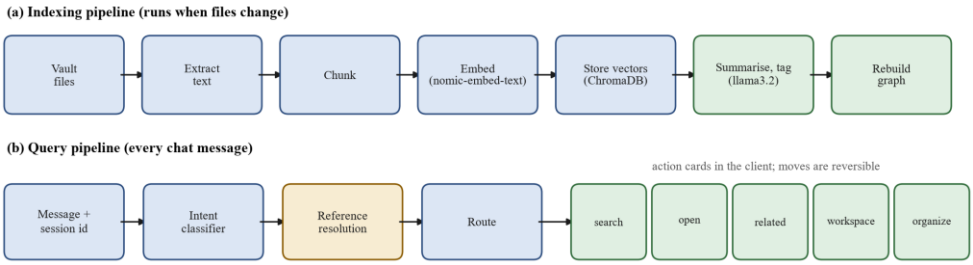


Fig. 2. (a) The indexing pipeline turns files into vectors, summaries, tags and graph edges. (b) The query pipeline classifies a message, resolves references (amber: the step that carries the paper's precedence rule), and routes to one of five actions.

C. Data model

Table II lists the design tables. Every derived table (edges, turns, usage, workspace membership) is keyed on the file's integer identifier, not its path. The organiser renames files when it moves them, but the identifier survives, so relationships and memory never need re-synchronising after a move, and a move can be reversed from the stored original path.

TABLE II. Database Design: Tables, Key Fields and Role

Store / table	Key fields	Role
SQLite / files	id (PK), path (unique), original_path, content_hash, summary, tags, category	One row per file; stable identity
SQLite / file_edges	src_id, dst_id, kind, weight	similar, references, shared_tag
SQLite / conversation_turns	session_id, role, message, intent, file_ids	Episodic memory of the chat
SQLite / usage_events	file_id, kind, created_at	Open history; tie-break and rules
SQLite / workspaces, workspace_files	name (unique); workspace_id, file_id	Named groups of files
ChromaDB / vectors	id, document chunk, embedding, metadata (path)	Nearest-neighbour retrieval

D. Trust mechanisms

A language model sits at four points where an error would cost the user something. Table III names each point, the mechanism that bounds it and the evidence reported in this paper. The mechanisms are deliberately simple and deterministic, so each can be tested on its own and explained to a user: none relies on the model grading itself.

TABLE III. Trust Mechanisms, the Failure Each Prevents and the Evidence

Mechanism	Failure it prevents	How it works	Evidence
Confidence guard on open	Launching a wrong file on a weak or ambiguous match	Opens only at similarity 0.55 or above with no runner-up within 0.05; otherwise shows candidates	Sec. VI-B: 29 opens, 0 wrong, against 2 wrong without it
Evidence before model	A stale or guessed reference overriding a clear request	Ordinal and pronoun rules, then a direct search; the model's flag is the last resort	Live failure; regression test fails on the old order (Sec. IV-C)
Reversible organisation	Irrecoverable moves of files	Move only on confirmation; original path stored; revert restores it	Apply-and-revert test; ids survive moves (Sec. III-C)
Checkable privacy audit	An unverified claim that data stays local	Lists every endpoint from live configuration; names a netstat check that can falsify it	Sec. IV-D; every endpoint is loopback

IV. METHODS

A. Semantic search

A query q is embedded and compared with every chunk c. A file f takes the score of its best chunk, plus a small boost for literal vocabulary overlap, where Q is the query's content words (stop-words removed, plurals folded) and T_f the words of the file's name and text:

$$s(f) = \max_c \cos(e_q, e_c) + \lambda \cdot |Q \cap T_f| / |Q| \quad (1)$$

$$\text{keep } f \text{ iff } s(f) \geq \max(\tau, s^* - \delta) \quad (2)$$

with floor $\tau = 0.47$, margin $\delta = 0.06$ below the best score s^*, and $\lambda = 0.08$. A fixed cut-off was abandoned because it dropped correct answers to short queries (20 of 27 on the tuning set, against 27 of 27 with the relative rule). Results below a confidence bar of 0.55 are labelled low-confidence, and the open action never launches a file from such a result.

B. Knowledge graph

Three typed, weighted edge kinds connect files: similar (cosine of mean chunk embeddings), references (one file's text names another by file name) and shared_tag (at least two AI-generated tags in common). Raw cosine is compressed by anisotropy, so vectors are first mean-centred, $\hat{h}_f = (h_f - \mu) / \|h_f - \mu\|$, which spreads unrelated pairs to about 0 and related pairs to 0.25-0.4; the similar-edge threshold is 0.25 on that scale and each file keeps its three strongest neighbours. Clusters use average-linkage merging, joining two groups only while the mean edge weight between their members, counting missing edges as 0, exceeds 0.12. Plain connected components chained unrelated groups through single weak bridges (a 7-file cluster mixing resumes with proposals); average linkage cannot chain.

The pairwise similarity loop was replaced by one matrix product, $S = \hat{H}\hat{H}^T$, and two further all-pairs steps by a token index (the reference check, which was 98% of the 5,000-file rebuild) and a heap over per-group adjacency (the clustering). Each was checked against its original definition on random data; ties in clustering are broken by lowest group id (Sec. VI-C).

C. Context memory and deterministic precedence

Each turn is stored with the identifiers of the files it showed, so follow-ups such as "open the second one" resolve after a restart or a file move. Fig. 3 gives the order: ordinal and bare-pronoun rules, then a direct search on the topic the classifier extracted, and only if that search is below the confidence bar the model's own "refers to previous" flag as a last resort.

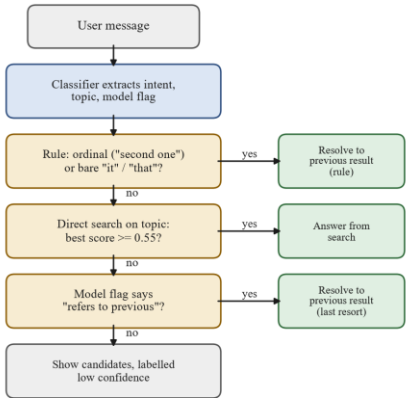


Fig. 3. Follow-up resolution flowchart. Amber: decisions, evaluated top to bottom; green: outcomes. Deterministic evidence precedes the language model's own judgement.

The order was fixed after a live rehearsal. Asked "what is related to my resume" after a search for invoices, the system answered about an invoice file. The 3B model set its continuation flag to true for this sentence while correctly

extracting "resume" as the topic, apparently because the sentence matched an example in its own instructions; the system had trusted the flag unconditionally. A regression test that fails on the old precedence and passes on the new guards it.

D. Privacy audit

An endpoint reads live configuration and lists every address the service is set to call, whether each is loopback, and what is stored locally. It states that it checks configuration, not traffic, and names the independent check (listing listening sockets with netstat), so the claim can be falsified.

E. Organiser and workspaces

For each file the model proposes a summary, tags, a category and a reason; a move is applied only on confirmation and is reversible. Workspace recommendations are rule-based on purpose, so they are reproducible, explainable and cost no model calls.

V. EXPERIMENTAL SETUP

All runs used one laptop (AMD Ryzen 7 6800H, 16 GB RAM, Windows 11, CPU only) with nomic-embed-text and llama3.2 3B on Ollama. The 123 automated tests run offline against fixtures and on every push in continuous integration.

A. Retrieval comparison

The corpus is the 37-file demonstration vault: deliberately messy text, Markdown and source files (CVs, invoices, meeting notes, a recipe, scripts). We wrote 36 queries with their relevant files before running any system. Each query is partitioned automatically by whether any of its content words, after the system's own stop-word and plural folding, occurs in a relevant file's name or text: 26 do, 10 do not. Four systems return at most 8 files: (i) filename match, a lexical match on names only, standing in for name-index tools [7]; (ii) BM25 [12] over name and text, standing in for lexical content indexes [5], [6]; (iii) embedding only, DreamOS without the keyword boost and cut-off; (iv) DreamOS as shipped. Metrics are Hit@1, Hit@3 and mean reciprocal rank (MRR), with 95% Wilson intervals [17] on Hit@1, following standard information-retrieval evaluation practice [18].

B. Scale evaluation

Synthetic vaults of 500, 2,000 and 5,000 text files across twelve topics (half with one distractor sentence from another topic) were each run in a fresh process with a fresh database and the real models, measuring indexing throughput, graph rebuild time, the latency of 36 queries and precision at 5 against the generated topic label. The scripts (bench_scale.py, eval_retrieval.py) are in the project repository.

TABLE IV. Retrieval Comparison on 36 Labelled Queries (37 Files)

System	Hit@1 (all)	Hit@3 (all)	MRR (all)	Hit@1, shares words (n=26)	Hit@1, no shared words (n=10)	MRR, no shared words
Filename match	0.33	0.36	0.35	0.46	0.00	0.00
BM25 over name and text	0.56	0.67	0.61	0.77	0.00	0.00
Embedding only	0.92	1.00	0.95	0.92	0.90	0.95
DreamOS (shipped)	0.94	1.00	0.97	1.00	0.80	0.90

VI. RESULTS

A. Retrieval quality

Table IV and Fig. 4 show the comparison. DreamOS places a relevant file first for 94% of all queries and within the first three for 100%, against 56% and 67% for BM25. The partition explains the gap: when a query shares a word with its target, BM25 reaches Hit@1 0.77 and DreamOS 1.00; when it shares none, both lexical baselines score exactly 0.00 by construction, while DreamOS scores 0.80 with all 10 targets inside the top three. Examples are "present for my mother" finding the gift list that says "Mom's birthday", and "how to brew a hot drink with spices" finding the chai recipe.

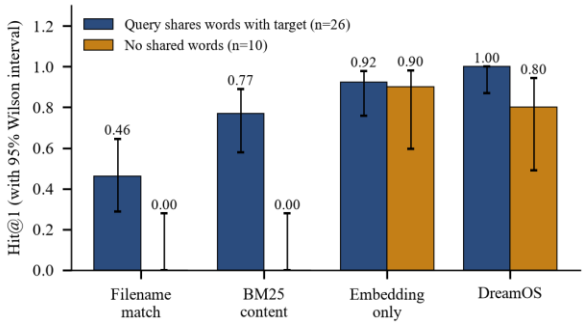


Fig. 4. Hit@1 by system and query group, with 95% Wilson intervals. Lexical systems cannot answer queries that share no word with the target; the embedding-based systems can.

Two findings are less favourable and are reported as measured. First, the keyword boost helps queries that share words (Hit@1 0.92 to 1.00) but costs one of the 10 paraphrase queries (0.90 to 0.80); the intervals overlap, so the data cannot separate the two configurations. Second, DreamOS missed rank 1 on two queries ("remarks from our mentor about what to improve" and "spending breakdown for the semester"), both of which placed an unrelated quotes file first with the correct file second. With 36 queries the Hit@1 interval for DreamOS is wide (about 0.82 to 0.99), so differences of a few points between the embedding systems should not be over-read.

B. Guarded actions

The open action is the one place where a retrieval error becomes an action on the user's computer, so we measured the confidence guard on the same 36 queries. Following the rule in Table III, a file is launched only when its similarity is at least 0.55 and no runner-up lies within 0.05. Opening the top result unconditionally would launch 36 files, 2 of them wrong. The guard launches 29 files, all 29 correct, and holds back 7 queries (2 below the bar, 5 with a close runner-up), showing candidates instead. Of the held-back queries, 5 would have opened the right file, so the price of the guard is 5 extra confirmations and the benefit is that both wrong opens are prevented. Two caveats apply: the bar of 0.55 was tuned earlier on this vault, so this is a measurement of the policy, not a held-out estimate; and the real application also resolves some close calls by the file the user opens most, which this offline evaluation does not model.

C. Scale

Table V and Fig. 5 report the scale runs. Indexing stays above 18.1 files per second up to 5,000 files and median search latency grows from 34.5 ms to 204.7 ms. Graph construction was sped up in two stages. Vectorising the similarity step cut the 500-file rebuild from 17.0 s to 1.7 s and the 2,000-file rebuild from 246.1 s to 24.8 s, but profiling the 5,000-file rebuild (143.3 s) showed that 98% of it was the reference check, an all-pairs regular-expression search. Replacing that with a token index, and the clustering with a heap over per-group adjacency, brought the rebuilds to 0.4 s, 1.3 s and 3.2 s at 500, 2,000 and 5,000 files, with the same edge set. Indexing is now dominated by embedding: at 5,000 files the graph is 1% of the 276.3 s index pass. Precision at 5 is 1.00 at every size, which shows only that the twelve synthetic topics separate; it is not a quality claim for real documents.

TABLE V. Scale Benchmark Across the Three Graph-Construction Stages (n/m: not measured)

Files	Index files/s (first / final)	Graph rebuild s (loop / vectorised / final)	Graph edges	Search p50 (ms)	Search p95 (ms)	Precision@5
500	12.4 / 19.9	17.0 / 1.7 / 0.4	1062	34.5	60.7	1.00
2,000	5.4 / 23.2	246.1 / 24.8 / 1.3	4747	106.5	126.7	1.00
5,000	n/m / 18.1	n/m / 143.3 / 3.2	11938	204.7	232.7	1.00

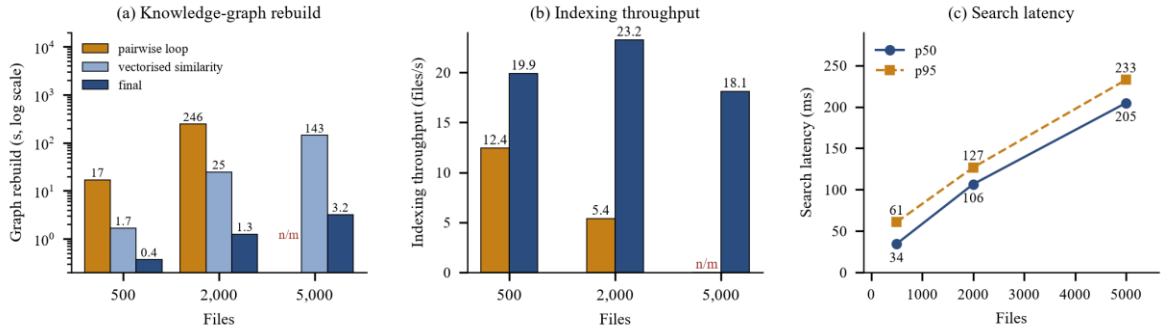


Fig. 5. Scale results from the same runs as Table V. (a) Graph rebuild time on a log scale at three stages: the original pairwise loop, vectorised similarity only, and the final version (token-indexed reference check, heap clustering). (b) Indexing throughput, which includes the graph rebuild. (c) Median and 95th-percentile search latency, which grows with the number of stored vectors.

D. Defects found by testing

Table VI lists the three defects that testing exposed. Each was found by measurement: the first by profiling, the second by scaling the input and observing super-linear growth, the third by a live rehearsal the unit tests had not anticipated. Each fix has a test that fails against the prior code.

E. Intent routing

The intent prompt was evaluated, not guessed: a 32-message tuning set and a separate 26-message held-out set on the real llama3.2 gave 80% held-out accuracy with the first prompt and 88% (23 of 26) with the revised one. The misses are borderline phrasings, for example "which files should I archive" routed to organise rather than workspace; every module sits behind this classifier, so the 12% bounds how often a request reaches the wrong module.

F. Interface

Fig. 6 shows three of the four views. Result cards carry an action and, for related files, the reason they are linked; the graph view is a force-directed layout of files and edges; the workspace view lists each recommendation with the rule that produced it.

TABLE VI. Defects Exposed by Testing, Their Causes and Fixes

Defect	Symptom	Cause	Fix and evidence
Name-resolution delay	About 1 s added to every embedding call	"localhost" tried IPv6 first; the model server listens on IPv4	Use 127.0.0.1; call about 15 ms; 100-file index 0.41 to 19.6 files/s
All-pairs graph computations	Rebuild 246.1 s at 2,000 and 143.3 s at 5,000 files	Pairwise similarity loop; then all-pairs regex reference check and full-table clustering rescan	Matrix product, token index, heap; equivalence tests; 3.2 s at 5,000
Stale-context override	Answer about the previous topic	Model flag trusted over a confident direct search	Rules first, flag last; regression test fails on old order



Fig. 6. Screenshots of the desktop client: (a) chat results with the reason each related file is linked, (b) the knowledge-graph view, (c) workspaces with rule-based suggestions.

VII. DISCUSSION AND THREATS TO VALIDITY

Meaning-based matching comes from the embedding model, not from the surrounding engineering; what DreamOS adds is one interface over retrieval, action and memory, and four trust mechanisms that are small enough to test and explain. Human-AI interaction guidelines [20] ask a system to make clear what it can do and how well, and to support efficient correction; the confidence labels, the guarded open action and the reversible moves implement those requirements in a way that can be measured (Tables III and IV, Section VI-B). The negative results matter as much: the keyword boost trades one paraphrase query for several shared-word ones, and the dense similarity matrix grows with the square of the file count (about 200 MB at 5,000 files), so beyond roughly ten thousand files an approximate index such as FAISS [19] or HNSW [15] would replace it.

Threats to validity. Internal: the authors wrote and labelled the queries; labels were fixed before the run, the partition is computed rather than judged, and failures are listed. The baselines are reference implementations of the lexical class, not the Windows Search, Spotlight or Everything products, so Table IV supports a claim about lexical versus embedding retrieval, not about any named product. Construct: precision on synthetic topic labels measures topic separation, not usefulness. External: 37 files, 36 queries, one vault, one machine, one pair of small models; intent routing is 88% correct on held-out phrasings. All numbers in this paper are read from committed result files produced by the two scripts above.

VIII. CONCLUSION

DreamOS shows that a single laptop with two small local models can support meaning-based file retrieval, conversational follow-ups and a relationship graph, while keeping the model's mistakes from becoming harmful actions and the privacy claim checkable. It retrieved paraphrased targets that lexical indexes cannot, its guarded open action launched no wrong file on the labelled queries, and measurement exposed a name-resolution delay, three all-pairs computations and one behavioural defect, each fixed and tested. Future work is a labelled evaluation on consented real documents, an approximate nearest-neighbour graph beyond 10,000 files, and a head-to-head run against the shipped desktop-search products.

ACKNOWLEDGMENT

The authors thank their project guide, Dr. Minakshi R. Rajput, and IICT, MGM University, for the Project Monitoring structure under which this work was reviewed. Generative AI (Claude, Anthropic) assisted with drafting text and with figure and analysis code; the authors checked all content and results and are responsible for them.

REFERENCES

[1] Z. Shi et al., “From Commands to Prompts: LLM-Based Semantic File System for AIOS,” in Proc. ICLR, 2025, arXiv:2410.11843.

[2] T. Mikolov, K. Chen, G. Corrado and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” arXiv:1301.3781, 2013.

[3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks,” in Proc. EMNLP-IJCNLP, 2019.

[4] Chroma, “Chroma: the open-source embedding database,” github.com/chroma-core/chroma.

[5] Microsoft, “Windows Search overview,” Microsoft Learn documentation.

[6] Apple Inc., “Spotlight User Guide,” support.apple.com.

[7] voidtools, “Everything: locate files and folders by name instantly,” voidtools.com.

[8] D. K. Gifford, P. Jouvelot, M. A. Sheldon and J. W. O'Toole, “Semantic file systems,” in Proc. 13th ACM Symp. on Operating Systems Principles (SOSP), 1991.

[9] V. Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering,” in Proc. EMNLP, 2020.

[10] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, Gaithersburg, MD, USA, 2023.

[11] S. Dumais, E. Cutrell, J. J. Cadiz, G. Jancke, R. Sarin and D. C. Robbins, “Stuff I've Seen: A System for Personal Information Retrieval and Re-Use,” in Proc. ACM SIGIR, 2003.

[12] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Found. Trends Inf. Retr., vol. 3, no. 4, 2009.

[13] K. Ethayarajh, “How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings,” in Proc. EMNLP-IJCNLP, 2019.

[14] J. Mu and P. Viswanath, “All-but-the-Top: Simple and Effective Postprocessing for Word Representations,” in Proc. ICLR, 2018.

[15] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, 2020.

[16] Z. Nussbaum, J. X. Morris, B. Duderstadt and A. Mulyar, “Nomic Embed: Training a Reproducible Long Context Text Embedder,” arXiv:2402.01613, 2024.

[17] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” J. Amer. Statist. Assoc., vol. 22, no. 158, 1927.

[18] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge University Press, 2008.

[19] J. Johnson, M. Douze and H. Jégou, “Billion-scale similarity search with GPUs,” IEEE Trans. Big Data, vol. 7, no. 3, 2021.

[20] S. Amershi et al., “Guidelines for Human-AI Interaction,” in Proc. ACM CHI Conf. on Human Factors in Computing Systems, 2019.